

CYBER SECURITY CTF MACHINE

DEVELOPMENT REPORT

COMPREHENSIVE BUILD DOCUMENTATION, HARDENING REVIEW, AND
VULNERABILITY EXPLOIT WALKTHROUGH

Prepared By:	Juveriya	Target System Static IP:	192.168.143.138
Department:	Department of Computer Applications	Configured Hostname:	juveriya
Date:	June 29, 2026	OS Baseline:	Ubuntu 26.04 LTS (x86_64)

1. Objective

The primary purpose of this project is to construct, configure, secure, and document an operational Capture The Flag (CTF) Linux laboratory instance. Designed from a defensive administration framework, the target sandbox architecture mimics real-world insecure deployments—specifically focusing on administrative oversights such as technical information leakage gaps, weak authentication mechanics, and local permission escalation paths. This project fulfills academic development metrics covering network interface isolation, firewall management policies, and interactive challenge verification chains.

2. Machine Specifications

To support high performance and isolated simulation reliability during testing phases, the target virtual environment was built using structured virtualization limits:

- **Hypervisor Environment Platform:** VMware Workstation Pro / vSphere Node Infrastructure
- **Virtual Processor Provisioning:** 2 vCPU Cores Architecture
- **Volatile Memory Base (RAM):** 4.0 GB Allocation Memory
- **Virtual Storage Target Size:** 25 GB Dynamically Allocated Storage Capacity

3. Operating System Used

The target environment runs an unhardened server installation of **Ubuntu 26.04 LTS (64-bit architecture)** operating on stable **Linux Kernel distribution baseline 7.0.0-14-generic x86_64**. Ubuntu was selected as the underlying system because it provides comprehensive control over security boundaries, user profiling structures, service rulesets, and native log monitors necessary to construct realistic vulnerability dependencies.

4. Network Configuration

To safely isolate exploit payloads within local testing boundaries, the target machine was assigned a single virtual network adapter interface. The configuration details gathered via terminal diagnostics include:

- **Active Adapter Card Interface:** ens33
- **Assigned Laboratory Static IP:** 192.168.143.138
- **Internal Subnet Broadcast Mask:** 255.255.255.0
- **Scope & Binding:** Broadcast multicast group configuration, statically mapped inside local testing networks to prevent IP drift while remaining completely isolated from standard external routing rules.

5. User Accounts Created

To fulfill deployment criteria requiring structural multi-user segmentation, the target system features four provisioned interactive user environments with localized home paths:

Username	Role Identity Type	Directory Environment Profile Path	Intended Security Purpose
ruhi	Low-Privilege Account	/home/ruhi/	Acts as a multi-stage puzzle pivot vector utilizing basic credentials discovered through steganography.
riya	Low-Privilege Account	/home/riya/	Maintained as an unprivileged profile environment to simulate lateral account complexity.
tina	Flag Custodian Profile	/home/tina/	Holds local milestone verification tokens for intermediate exploitation validation.
juveriya	High-Privilege Admin User	/home/juveriya/	Global administrative environment maintaining full vertical elevation and local command privileges.

6. Services Configured

To optimize performance and minimize background signature noise, only two active services were exposed on the external interfaces:

- **Secure Shell Service Daemon (SSH):** Running OpenSSH Server Engine on standard **Port 22** to handle remote session entries and system access validation.
- **HTTP Web Daemon:** Driven via the **Apache2 HTTPD Server** engine on standard **Port 80** to serve web-based platform assets.

7. Website Design Overview

The front-end web portal interface hosted under `/var/www/html/` features a professional developer profile portfolio. It highlights academic background steps, standard development skills, active industry certifications, and completed repository projects. The content mimics a production developer platform to cleanly blend with the challenge elements hidden within its code directories.

8. Hidden Credentials Methodology

To reward thorough web directory inspection, player access tokens are split into separate stages instead of being exposed directly on the root landing page:

- **The Disallowed Endpoint:** The system maps an active `robots.txt` parameter file pointing to isolated directories to guide scanners away from sensitive paths while inadvertently creating an enumeration target.
- **The Directory Leak String:** Accessing files within the web server layout exposes hint structures designed to direct the tester to look into file metadata matrices and local system accounts.

9. Steganography Implementation

An image asset named `photo.jpg` was pulled onto the server directory. This asset functions as a core challenge checkpoint. Using `steghide`, a hidden credential file named `resume.txt` was embedded inside the pixel matrix.

- **Payload Contents:** **"This is my hidden resume! Hint: ruhi password is 'ruhi'"**
- **Extraction Requirement:** To extract it, analysts use credential mapping techniques discovered during initial system asset enumeration, testing their file carving and metadata analysis skills.

10. Flag Locations

- **User Milestone Flag Target:** Mapped at path `/home/tina/user.txt`

Token Value String: **CTF{you_found_the_user_flag}**

- **Administrative Privilege Root Flag:** Mapped at path /root/root.txt

Token Value String: **CTF{you_are_the_ultimate_root_hacker}**

11. Screenshots of Configuration

Asset Staging & Baseline Configuration

```
juveriya@juveriya:~$ wget -O photo.jpg https://picsum.photos/500/500
--2026-06-29 11:21:21-- https://picsum.photos/500/500
Resolving picsum.photos... 104.26.5.30, 172.67.74.163
Connecting to picsum.photos|104.26.5.30|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://fastly.picsum.photos/id/936/500/500.jpg [following]
Length: 28900 (28K) [image/jpeg]
Saving to: 'photo.jpg'

photo.jpg          100%[=====] 28.22K  --.-KB/s    in 0.06s
```

Figure 1: Retrieving image components (photo.jpg) for the steganographic challenge base.

```
(root@kali)-[/home/kali]
# ssh juveriya@192.168.143.138
The authenticity of host '192.168.143.138' can't be established.
ED25519 key fingerprint is SHA256:J5QnAw1j0YL7EZLdZToAJ4f9iC83gD+HH1zgg8G0/qg
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.143.138' to the list of known hosts.
juveriya@192.168.143.138's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-14-generic x86_64)
```

Figure 2: Verifying initial system deployment signatures and SSH reachability metrics.

```
juveriya@juveriya:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script...
juveriya@juveriya:~$ sudo systemctl start apache2
juveriya@juveriya:~$ sudo nano /var/www/html/index.html
juveriya@juveriya:~$ sudo nano /var/www/html/robots.txt
juveriya@juveriya:~$ sudo apt install steghide -y
Installing: steghide
Installing dependencies: libjpeg-turbo8 libjpeg8 libmcrypt4 libmhash2
```

Figure 3: Provisioning Apache2 services, configuring robots.txt configurations, and setting up steghide deployment binaries.

Flag Deployment & System Hardening

```
juveriya@juveriya:~$ echo "CTF{you_found_the_user_flag}" | sudo tee /home/tina/
user.txt
CTF{you_found_the_user_flag}
juveriya@juveriya:~$ sudo chown tina:tina /home/tina/user.txt
juveriya@juveriya:~$ sudo mkdir -p /root
juveriya@juveriya:~$ echo "CTF{you_are_the_ultimate_root_hacker}" | sudo tee /root/
root.txt
CTF{you_are_the_ultimate_root_hacker}
juveriya@juveriya:~$ sudo chmod 600 /root/root.txt
juveriya@juveriya:~$ sudo apt install apache2 -y
```

Figure 4: Generating low-privilege and high-privilege validation flags with explicit ownership separation rules.

```
(kali@kali)-[~]
$ sudo su
[sudo] password for kali:
(root@kali)-[/home/kali]
# ping 192.168.143.138
PING 192.168.143.138 (192.168.143.138) 56(84) bytes of data.
64 bytes from 192.168.143.138: icmp_seq=1 ttl=64 time=1.79 ms
64 bytes from 192.168.143.138: icmp_seq=2 ttl=64 time=1.82 ms
64 bytes from 192.168.143.138: icmp_seq=3 ttl=64 time=1.80 ms
^C
--- 192.168.143.138 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 1.630/1.841/2.162/0.174 ms
```

Figure 5: Executing ICMP verification sweeps from the attack platform to validate target system up-status.

```
juveriya@juveriya:~$ sudo adduser ruhi
New password:
Retype new password:
passwd: password updated successfully
juveriya@juveriya:~$ sudo adduser riya
New password:
Retype new password:
passwd: password updated successfully
```

Figure 6: Provisioning target accounts ruhi and riya into local /etc/passwd environments.

```
juveriya@juveriya:~$ echo "This is my hidden resume! Hint: ruhi password is 'ruhi'" > resume.txt
juveriya@juveriya:~$ steghide embed -ef resume.txt -cf photo.jpg
Enter passphrase:
Re-Enter passphrase:
embedding "resume.txt" in "photo.jpg" ... done
juveriya@juveriya:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
juveriya@juveriya:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
juveriya@juveriya:~$ sudo ufw allow 22/tcp
juveriya@juveriya:~$ sudo ufw allow 80/tcp
juveriya@juveriya:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
```

Figure 7: Embedding text-based clues into photo.jpg via steghide and implementing structural firewall rules via UFW.

```
juveriya@juveriya:~$ sudo adduser tina
[sudo] password for juveriya:
New password:
Retype new password:
passwd: password updated successfully
Is the information correct? [Y/n] y
```

Figure 8: Creating user profile tina to complete target environment configuration blueprints.

```
juveriya@juveriya:~$ ip a
1: lo: mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
2: ens33: mtu 1500 qdisc pfifo_fast state UP group default
    link/ether 00:0c:29:18:06:3e brd ff:ff:ff:ff:ff:ff
    inet 192.168.143.138/24 brd 192.168.143.255 scope global dynamic ens33
```

Figure 9: Summary readout of interface mapping, kernel compilation details, and memory limits.

12. Testing Procedure

Sanity tests were executed by running black-box scans and connectivity diagnostics against the host from a remote testing machine. First, firewall confirmation routines verified that attempts to connect via unapproved ports are dropped by default incoming policy structures. ICMP sweep testing confirmed stable latency times averaging around 1.84 ms, validating that service daemons operate unimpeded by system resource exhaustion or configuration blockages.

13. Vulnerability Walkthrough

The complete exploitation path proceeds through four distinct stages:

- 1. **Information Gathering:** A remote diagnostic check determines that port 22 (SSH) and port 80 (HTTP) are exposed through the firewall matrix.
- 2. **Web Enumeration:** Accessing the web root reveals structural elements including a non-indexed robots.txt file and target directory hints pointing to user profiles.
- 3. **Steganography Decoding & Foothold:** Extracting the embedded file from photo.jpg using standard utilities reveals cleartext hints: the password for account ruhi. Logging in provides a remote terminal foothold to laterally access user milestone flag assets stored under /home/tina/user.txt.
- 4. **Privilege Escalation:** Capitalizing on configuration gaps or administrative command vectors available within the sudo configuration environment allows an attacker to elevate permissions, navigating straight to the root storage directory to uncover the administrative flag at /root/root.txt.

14. Risk Analysis and Mitigation Matrix

Identified System Flaw	Risk Rating	Technical Mitigation Action Strategy
Cleartext Authentication Clues in Hidden Assets	CRITICAL	Remove descriptive hint text files from public web server paths; integrate system verification routines with centralized vault management tools.
Sensitive Endpoint Disclosure via Public Directives	HIGH	Mask developmental files or test credentials from public robots.txt tracking; restrict internal access bounds using directory-level access controllers.
Password-Based Authentication Weakness	MEDIUM	Enforce public-private key exchange authentication frameworks on SSH service instances; deactivate plaintext password authentication mechanisms entirely.

15. Conclusion

The deployed laboratory environment successfully meets all project guidelines. By separating user accounts, embedding indicators in image matrix layers, and enforcing structural security parameters via firewall rule layers, the platform serves as a viable sandbox. It requires analysts to perform network mapping, target enumeration, asset content parsing, and privilege elevation vectors to compromise the architecture safely.